

Regression Trees and Ensemble Methods

David Puelz

The University of Austin

Beyond Linearity



Every model we have built assumes a particular functional form:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$$

LASSO and ridge select or shrink predictors—but still assume *linearity* in each retained variable.

Even with polynomials, splines, and interactions, we choose the structure.

What if the data could discover the structure on its own?

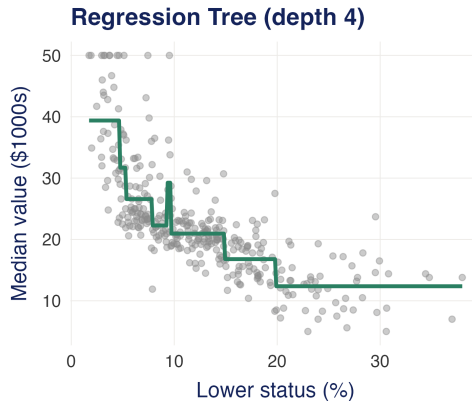
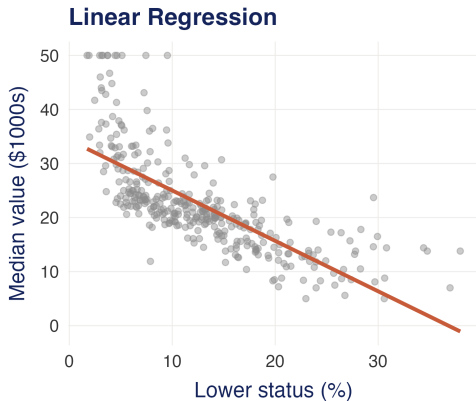


Goal: Predict median home values (`medv`) in Boston suburbs.

Data: 506 census tracts, 13 predictors:

- `lstat` — % lower-status population
- `rm` — avg. rooms per dwelling
- `crim`, `age`, `dis`, `nox`, `tax`, ...

`medv` vs. `lstat` is strongly nonlinear—a straight line misses the pattern.



The tree adapts its shape to the data without us specifying the functional form.



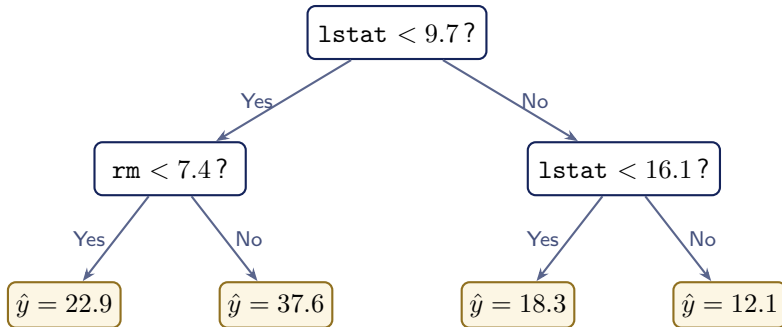
Instead of fitting a global function:

1. **Partition** the predictor space into J distinct regions $R_1, \dots, R_J$
2. **Predict** the mean of Y in each region: $\hat{y} = \bar{y}_{R_j}$

The regions are found by *recursive binary splitting*—a series of yes/no questions about the predictors.

This is a **regression tree**.

Regression Trees



Internal nodes: splitting rules. **Leaves:** predicted values (region averages).

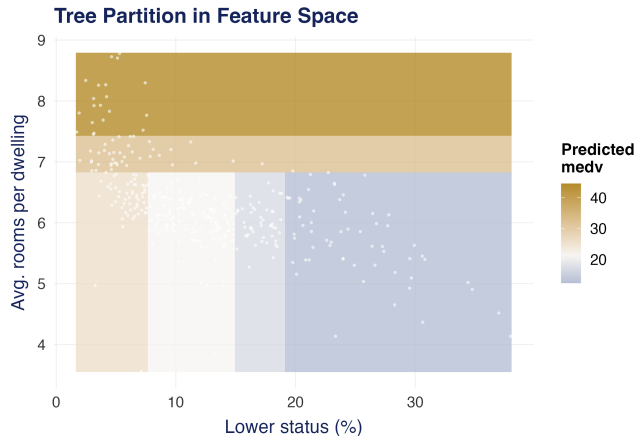


At each step, pick variable X_j and split point s to minimize

$$\sum_{i: x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2$$

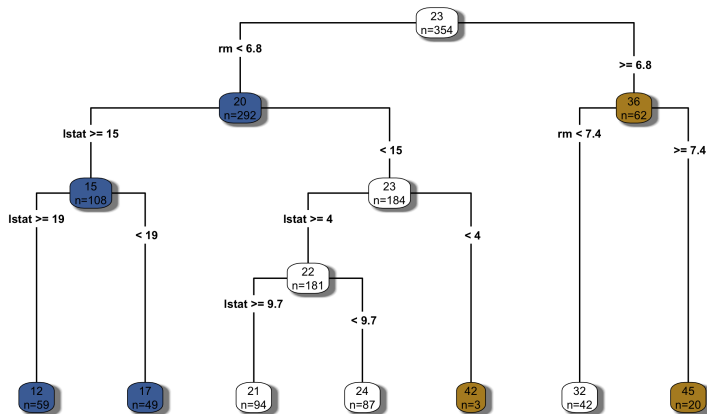
where $R_1(j, s) = \{X : X_j < s\}$ and $R_2(j, s) = \{X : X_j \geq s\}$.

- This is **greedy**: best split *right now*, no look-ahead
- Repeat recursively within each region
- Stop when a region has too few observations or no improvement

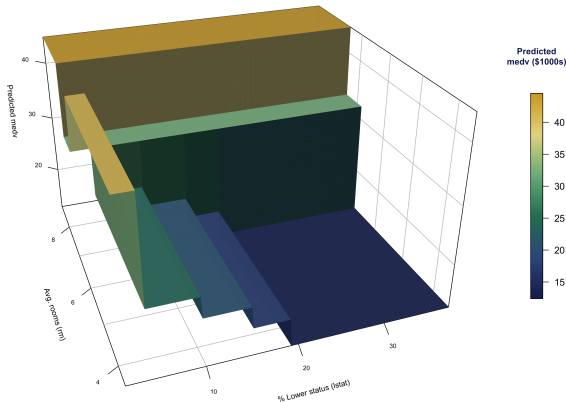


Each split creates an axis-aligned boundary. The tree carves the space into rectangles.

A tree on two predictors

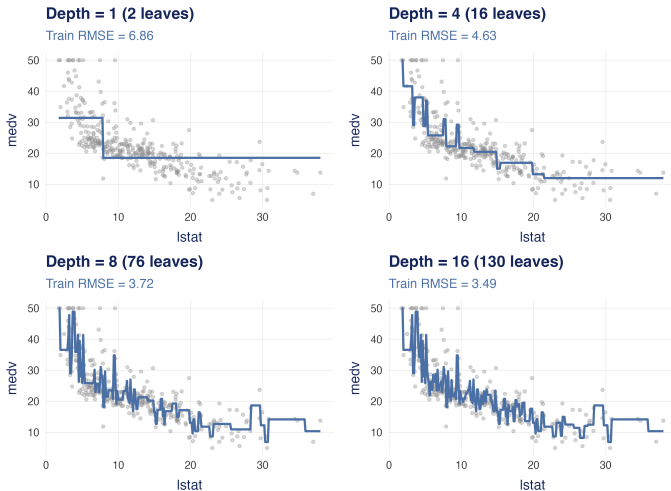


A depth-4 tree using only `rm` and `lstat`. Notice that `rm` appears at multiple levels—the effect of rooms *depends on* the neighborhood's socioeconomic status.

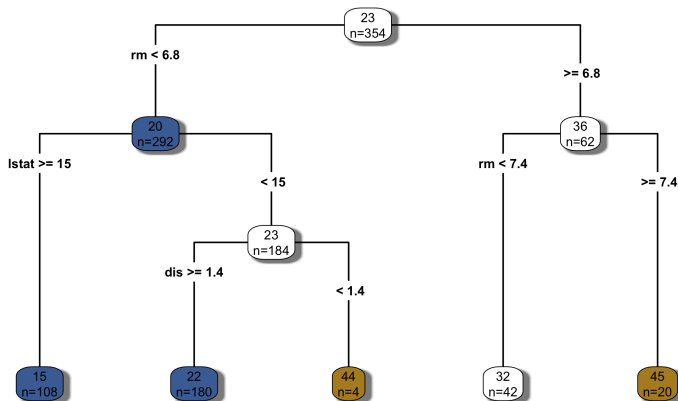


The same tree as a **piecewise-constant surface**—flat plateaus separated by sharp jumps. The rectangular regions **automatically capture interactions**: high *rm* matters most when *lstat* is low. No interaction terms needed.

Tree depth controls complexity



Depth 1–2: underfitting. Depth 4: balance. Depth 8–16: overfitting.



Pruned tree with 5 terminal nodes. The two most important splits are on `rm` and `lstat`.



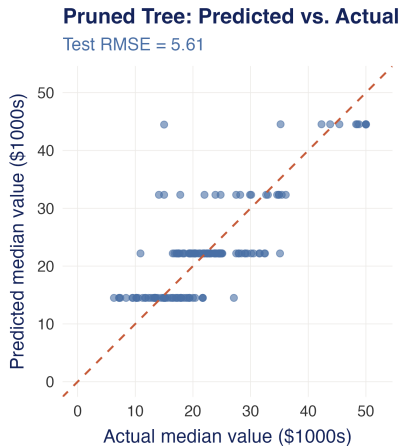
```
library(rpart)
library(rpart.plot)

# Fit a regression tree
tree ← rpart(medv ~ ., data = train)

# Visualize the tree
rpart.plot(tree, type = 4, extra = 1)

# Predictions on test set
pred ← predict(tree, test)
sqrt(mean((pred - test$medv)^2))    # Test RMSE
```

How well does the tree predict?



Reasonable, but the step-function nature of trees limits precision.

Pruning

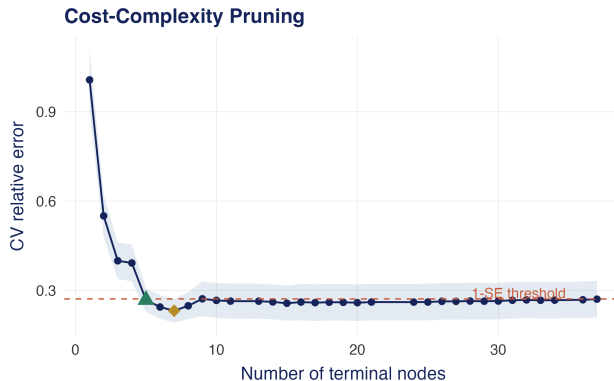


A deep tree has **low bias** but **high variance** — it memorizes the training data.

Cost-complexity pruning finds the right tradeoff:

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|$$

- $|T|$ = number of terminal nodes (tree size)
- $\alpha \geq 0$ penalizes complexity (like λ in LASSO!)
- Larger $\alpha \Rightarrow$ smaller tree $\Rightarrow$ more bias, less variance



Gold diamond: min CV error. Green triangle: 1-SE rule. Full tree: 37 leaves $\rightarrow$ Pruned: **5 leaves**.



```
# Grow a large tree
big_tree ← rpart(medv ~ ., data = train,
                  control = rpart.control(cp = 0.001))

# 1-SE rule: simplest tree within 1 SE of min
cp_tab ← big_tree$cptable
min_idx ← which.min(cp_tab[, "xerror"])
thr ← cp_tab[min_idx, "xerror"] + cp_tab[min_idx, "xstd"]
best_cp ← cp_tab[min(which(cp_tab[, "xerror"] <= thr)), "CP"]

pruned ← prune(big_tree, cp = best_cp)
rpart.plot(pruned, type = 4, extra = 1)
```



Strengths

- Highly interpretable
- Handle nonlinearity naturally
- No need to scale predictors
- Handle mixed variable types

Weaknesses

- High variance (unstable)
- Axis-aligned splits only
- Greedy: no global optimum
- Outperformed by ensembles

The main enemy is **variance**. Can we keep the strengths while taming the variance?

Bagging



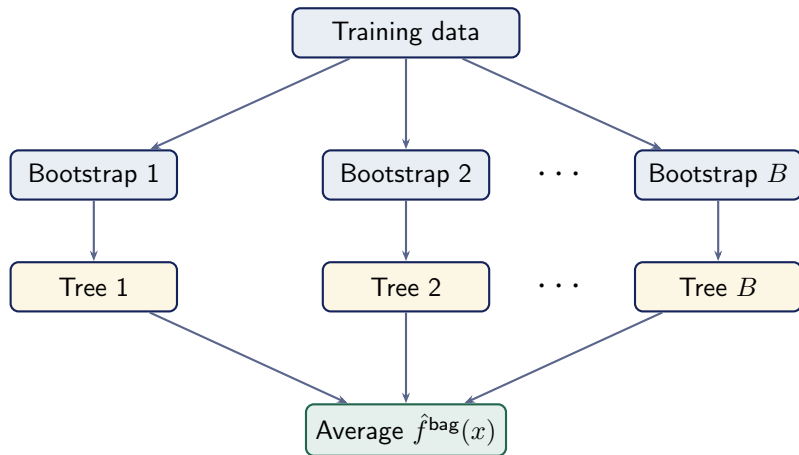
Trees are **high-variance** estimators.

Small changes in the training data $\Rightarrow$ very different trees.

Recall from statistics: if $Z_1, \dots, Z_B$ are i.i.d. with variance σ^2 :

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{B}$$

Averaging reduces variance. Can we average trees?



Draw B bootstrap samples $\rightarrow$ fit a deep tree to each $\rightarrow$ average the predictions.



Bagging for regression trees

1. For $b = 1, \dots, B$:
 1. Draw a bootstrap sample $\mathbf{Z}^{*b}$ from the training data
 2. Fit a deep, unpruned tree $\hat{f}^b$ to $\mathbf{Z}^{*b}$
2. Predict: $\hat{f}^{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x)$

Each tree is **high variance**, **low bias**. Averaging keeps the low bias while reducing variance. Typical B : 100–500 trees. More is better (never overfits by adding trees).

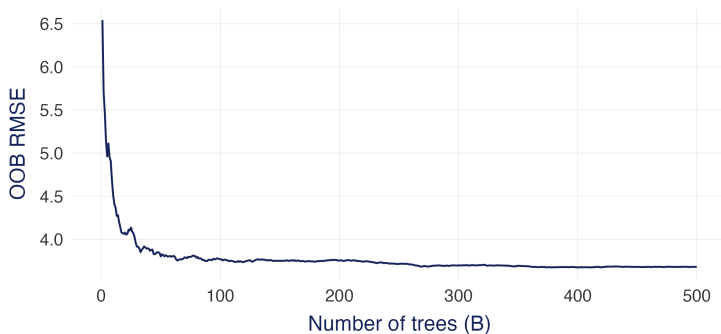


Each bootstrap sample uses $\approx 63\%$ of observations. The remaining $\approx 37\%$ are **out of bag (OOB)**.

Key insight: For each observation i , average predictions from only the trees that *did not* use i in training. $\Rightarrow$ Free cross-validation!

Bagging: OOB Error Converges as B Grows

All 13 predictors used at each split ($m_{\text{try}} = p$)



More trees never hurts—OOB error converges. Typical: $B = 300$ – 500 .



```
library(randomForest)

# Bagging = randomForest with mtry = p
bag <- randomForest(medv ~ ., data = train,
                    mtry = 13,          # all 13 predictors
                    ntree = 500,
                    importance = TRUE)

# OOB RMSE (built-in cross-validation)
sqrt(tail(bag$mse, 1)) # 3.68

# Test RMSE
pred_bag <- predict(bag, test)
sqrt(mean((pred_bag - test$medv)^2)) # 3.59
```

Random Forests



Bagging averages B trees from bootstrap samples. But:

- If one predictor dominates (e.g., 1stat), **every tree** uses it for the first split
- The B trees are **correlated**
- Averaging correlated quantities gives less variance reduction

The fix: At each split, randomly select m out of p predictors. Only consider those m for the split.

This **decorrelates** the trees.

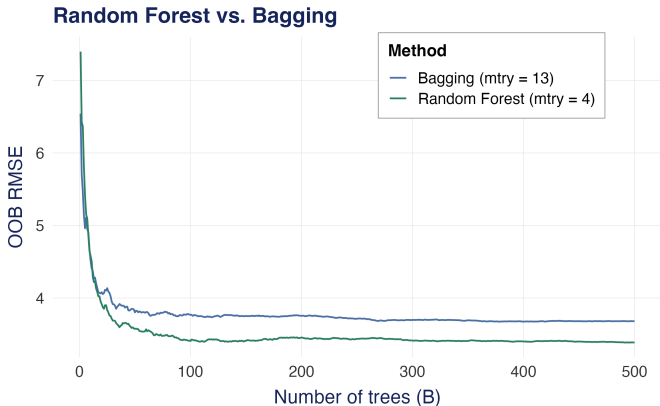


Simplified model: B random variables, each with variance σ^2 and pairwise correlation ρ :

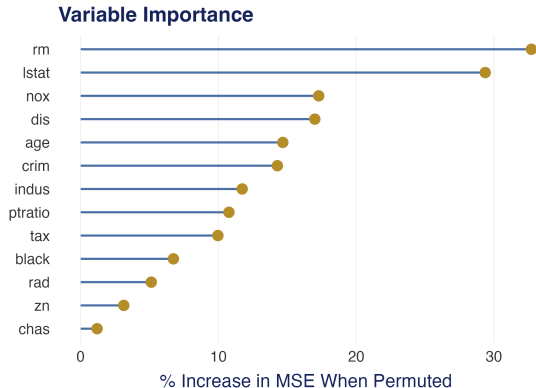
$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B Z_b\right) = \rho\sigma^2 + \frac{1-\rho}{B} \sigma^2$$

- As $B \rightarrow \infty$, the second term vanishes
- But the first term $\rho\sigma^2$ **persists**
- Reducing ρ (via random feature selection) directly reduces variance

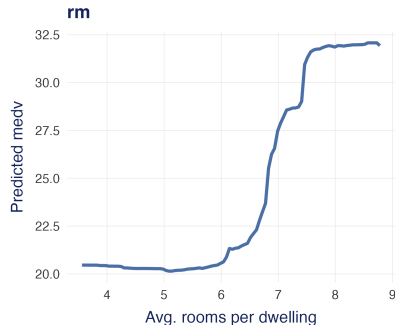
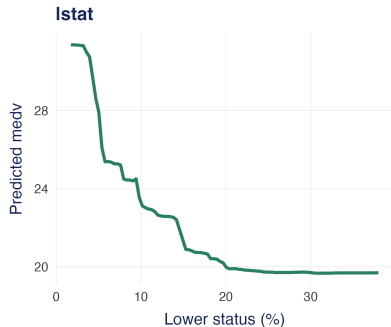
Typical m : $m \approx p/3$ for regression, $m \approx \sqrt{p}$ for classification.



Random forests ($mtry = 4$) achieve lower error than bagging ($mtry = 13$) by decorrelating the trees.



Permute each variable, measure MSE increase. Larger = more important. `lstat` and `rm` dominate.



Marginal effect of each variable, averaging over all others. `lstat`: negative nonlinear. `rm`: positive.



```
# Random forest: mtry < p decorrelates trees
rf ← randomForest(medv ~ ., data = train,
                  mtry = 4,             # p/3 ~ 4
                  ntree = 500,
                  importance = TRUE)

# OOB RMSE
sqrt(tail(rf$mse, 1))    # 3.39

# Variable importance
importance(rf)
varImpPlot(rf)

# Typically requires very little tuning!
```

Boosting



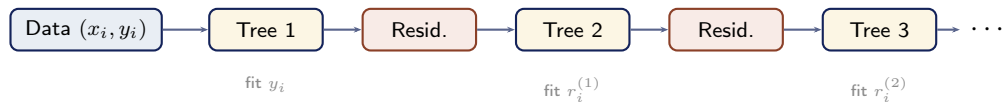
Bagging / RF

- Build many trees **independently**
- Average their predictions
- Reduces variance

Boosting

- Build trees **sequentially**
- Each tree learns from the *mistakes* of the previous ones
- Reduces bias

Boosting grows **small, weak** trees (often just stumps) and combines them into a strong learner.



Each new tree fits the **residuals** left by the previous model, slowly correcting mistakes.

Final prediction: $\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x)$ where λ is the **learning rate**.



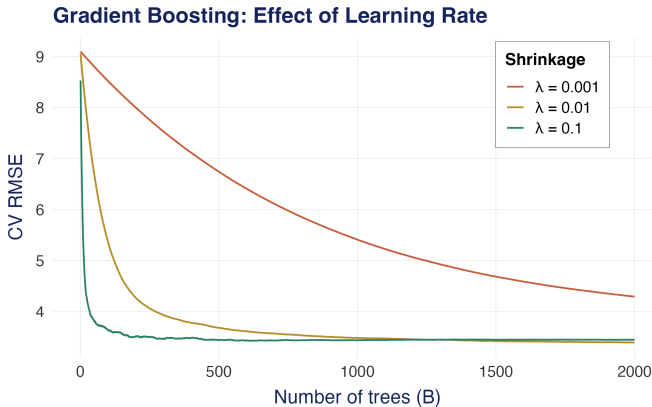
Gradient Boosting for Regression

1. Set $\hat{f}(x) = 0$ and residuals $r_i = y_i$ for all i
2. For $b = 1, \dots, B$:
 1. Fit a tree $\hat{f}^b$ with d splits to the residuals (x_i, r_i)
 2. Update the model: $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$
 3. Update residuals: $r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$
3. Output the boosted model: $\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x)$



Three knobs to turn:

1. B (number of trees): Unlike bagging, boosting **can overfit** with too many trees. Select via cross-validation.
2. λ (shrinkage / learning rate): Controls how fast the model learns.
 - Small λ (e.g., 0.01): slow, needs more trees, often better
 - Large λ (e.g., 0.1): fast, fewer trees needed, risk of overfitting
3. d (interaction depth): Number of splits per tree.
 - $d = 1$: stumps (additive model, no interactions)
 - $d = 2-4$: captures moderate interactions



Smaller λ requires more trees but often finds a better solution.



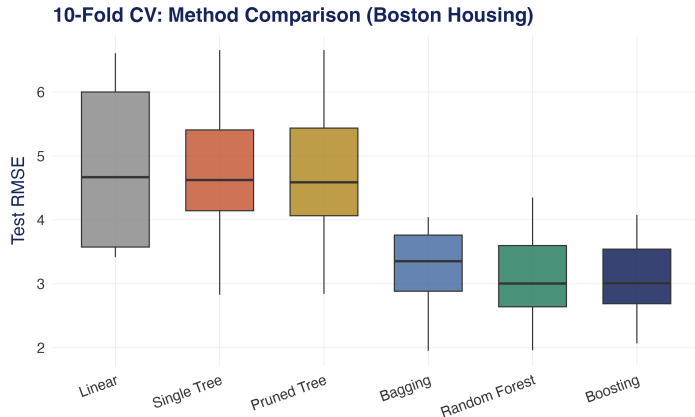
```
library(gbm)

boost ← gbm(medv ~ ., data = train,
            distribution = "gaussian",
            n.trees = 5000,
            interaction.depth = 4,
            shrinkage = 0.01,
            cv.folds = 5,
            verbose = FALSE)

# Optimal number of trees via CV
best_n ← gbm.perf(boost, method = "cv")

# Predictions
pred ← predict(boost, test, n.trees = best_n)
sqrt(mean((pred - test$medv)^2)) # Test RMSE
```

Putting It All Together



Ensemble methods (bagging, RF, boosting) consistently outperform single models.



Method	Mean RMSE	SD
Linear Regression	4.78	1.31
Single Tree	4.67	1.10
Pruned Tree	4.65	1.13
Bagging	3.20	0.72
Random Forest	3.14	0.77
Boosting	3.07	0.68

Ensembles reduce test RMSE by $\sim 30\%$ compared to linear regression.

RF and bagging are nearly tied; boosting is sensitive to tuning and can match or beat RF with careful optimization.

	Interpretability	Tuning effort	Accuracy
Linear Regression	High	Low	Moderate
Single Tree	High	Low	Moderate
Bagging	Low	Low	High
Random Forest	Low	Low	High
Boosting	Low	Medium	Highest

- Need to **explain** predictions? $\Rightarrow$ Single tree or linear model
- Need best **out-of-the-box** accuracy? $\Rightarrow$ Random forest
- Willing to **tune** for the best performance? $\Rightarrow$ Boosting



1. **Trees** partition the feature space into regions and predict averages—interpretable but unstable
2. **Pruning** controls tree complexity via a cost-complexity penalty α chosen by CV
3. **Bagging** averages many bootstrap trees to reduce variance
4. **Random forests** decorrelate trees by randomly restricting which predictors can be used at each split
5. **Boosting** builds trees sequentially, fitting residuals with a small learning rate
6. **Ensembles** consistently outperform single models, trading interpretability for accuracy
7. Everything here extends to **classification** (Gini index, majority vote)—same ideas, different loss function